

Assignment 2

Due Wednesday, March 5

The assignment contains three parts. Part I (the common part for both 4MA433 and 4MA923) consists of a written part and an implementations part. Part II (only for 4MA433) is the theoretical part. It contains more theoretical problems sometimes depending on knowledge of abstract algebra. Part III (only for 4MA923) is the computational part. It contains more advanced programming problems.

- It is fine to work together as long as you write and understand your own solutions. (No copying!)
- To each assignment each student should submit three files on Moodle:
 1. Part I (written part) as a pdf-file. You can use L^AT_EX for this part, but it is also fine to submit scanned handwritten solutions. You are responsible for these being readable.
 2. Part I (implementation part) as a Python file (.py) or a Jupyter file(.ipynb)
 3. Students in 4MA433 submit Part II as a pdf-file. Students in 4MA923 submit Part III as a Python- or Jupyter-file.
- On the seminars the problems of the Part I is presented and discussed. You need to understand how the algorithms work and be able to motivate your implementations. Part II and Part III are discussed on the oral exam in the end of the course.
- Part I is graded according to a linear scale. To pass (grade E) 50% is needed, for grade C, 70% is needed and for grade A, 90%. Part II and Part III is graded together with the oral exam.

Part I (common part)

Implementation part

1. We will now start looking for large prime numbers that we can use for setting up an RSA system. You can for example use the implementations `millerRabinWitnessQ(a,n)`, `probablePrimeQ(p)` and `findPrime(lBound,uBound)` described in the end of this document.
 - (a) Try the implementations out and find a probable prime with 16 digits. Check also that it really is a prime.
 - (b) Find a (probable) prime with 256 binary digits. (You do not have to check this, we hope for the best.)
2. We will now implement the RSA cryptosystem. You have to use the functions `findPrime`, `fast2Power`, `textToInt` and `intToText`
 - (a) Construct a function `generateRSA(b)` where b is the size in bits of the prime numbers. This function should output the following five parameters:
 - Two prime numbers p and q with b bits each.
 - The product $N = pq$.
 - An encryption exponent e that is relatively prime to $\phi(N)$.

- A decryption exponent d that is the inverse of e modulo $\phi(N)$.

Hint: To find e , you can generate a random integer and check that it is relative prime to $\phi(N)$. You could for example use something similar to the following piece of code (where you replace `emodulus` by an expression for $\phi(N)$):

```

1 while True:
2     e=randint(2,emodulus-1)
3     g=extendedGCD(e,emodulus)
4     if g[0]==1:
5         d=g[1]%emodulus
6         break

```

- Write the encryption- and decryption functions `rsaEncrypt(N,e,m)` (where m is the encoded message) and `rsaDecrypt(p,q,d,c)` where c is the ciphertext.
- Generate an RSA system where the primes have 128 bits each. Use the public keys of the system to encrypt a message (of at most 31 characters). Use the private keys to decrypt it. Did you recover the message?
- I have used an RSA system with the parameters

$N = 105114847348314514585994955084750533038746346509989025403991269864291488981373$,
 $p = 328496374651985341751351403397841563687$,
 $q = 319987846014054113870502192171918312379$,
 $e = 59003398073203884771435452033069769026412383780213144706044947972291416680509$,
 $d = 100731395982798615409977879267539034225873757936583625030901348545786440176489$.

Decrypt the ciphertext

48179554296613577325435211371310635867298025234458332739257346791666502996498

Where are we?

- In last homework we talked about Safe primes. These are primes p of the form $p = 2q + 1$, where also q is a prime.
 - Write a function `findSafePrime(lBound,uBound)` that finds primes q and $p = 2q + 1$ such that q is between `lBound` and `uBound`. Use the functions `findPrime` and `ProbablePrimeQ`.
 - Find a safe prime $p = 2q + 1$ with 128 bits and where q also is a prime.
 - Find a primitive root g modulo p . Find also an integer g' with order q .
 - Write a function `generateSafeElgamal(bits)` that generates the parameters for an Elgamal cryptosystem. Input is the number of bits for the prime p (which should be a Safe prime). Output should be p , g (an element of large prime order), a (the private key) and A (the public key).
- Implement Pollard's $p - 1$ factorization method. Write a function `factorPollard(N,a,m)`, where N is the number to factor, a is a number $1 < a < N$, and m is the limit for the factorial. Hence, $a^m!$ is the largest factorial what will be computed. Your function should give an appropriate answer if $\gcd(a^{k!} - 1, N) = N$ for some k or if it stops before having found a factor. *Hint:* Use the `fast2Power+` function and the fact that $k! = (k - 1)! \cdot k$.

5. Factor the following number by using Pollard's $p - 1$ -method.
 - (a) $N = 7648424392789$
 - (b) $N = 76504067652056510826705773878201$
6. Implement a function `factorPollardRho(N,s=2)` that uses Pollard's ρ -method for factorization of the positive integer N . The parameter s is the starting value ($x_0 = y_0 = s$) for the two lists $x_{i+1} = f(x_i)$ and $y_{i+1} = f(f(y_i))$. You can use $f(x) = x^2 + 1 \pmod{N}$.
7. Factorize the following numbers by using Pollard's ρ -method
 - (a) 4171,
 - (b) 43157855252213089,
 - (c) 2377101051469099400256636683.
8. We will now implement the Pollard ρ -algorithm for finding discrete logarithms. Write the function `discretePollardRho(g,h,p)` that search for a solution to $g^x \equiv h \pmod{p}$. Use the function `solveLinear(a,b,m)` that solves the linear congruences $ax \equiv b \pmod{m}$. You can find it in the end of this document.
9. Use the Pollard ρ -method for finding
 - (a) $\log_g(h)$ for $g = 19$, $h = 24717$, $p = 48611$,
 - (b) $\log_g(h)$ for $g = 29$, $h = 4417601$ och $p = 5272643$

Part II (Theoretical part)

Only for students in 4MA433.

1. Let p and q be odd prime numbers, and let $n = pq$. Let $\lambda(n)$ be Carmichael's totient function. It is the smallest positive integer m such that $a^m \equiv 1 \pmod{n}$ for every a relatively prime to n . One can prove that $\lambda(pq) = \text{lcm}(\phi(p), \phi(q))$, where ϕ is Euler's totient function and lcm denotes the least common multiple. We will now investigate this in detail.

- (a) Show that

$$\text{lcm}(x, y) = \frac{xy}{\text{gcd}(x, y)}$$

for all positive integers x and y .

- (b) Let a be an integer such that $\text{gcd}(a, pq) = 1$. Prove that

$$a^{\frac{(p-1)(q-1)}{\text{gcd}(p-1, q-1)}} \equiv 1 \pmod{pq}.$$

Hint: Prove first that the above congruence is true modulo p and modulo q .

- (c) Let e be the encryption exponent for an RSA system. Show that you can use the inverse $\tilde{d}$ of e modulo $\lambda(pq)$ as the decryption exponent.

2. Let p be a prime number such that $p \equiv 3 \pmod{4}$. Let $\gcd(a, p) = 1$. Find an explicit formula for solving $x^2 \equiv a \pmod{p}$ if you know the equation has solutions. Prove your formula is correct. *Hint:* If we know the equation has solutions then $a^{(p-1)/2} \equiv 1 \pmod{p}$.
3. Let p be an odd prime and let $\gcd(a, p) = 1$. Assume that the equation $x^2 \equiv a \pmod{p}$ has the solutions x_1 and y_1 modulo p .
 - (a) Prove that $x^2 \equiv a \pmod{p^e}$ also have two incongruent solutions modulo p^e for any $e \geq 1$. Do this by an recursive argument: If you have the solutions modulo p^e construct the solutions modulo p^{e+1} .
 - (b) Find all the solutions of $x^2 \equiv 2 \pmod{7^3}$.
4. Explain how Problem 2 and Problem 3 can be used in the Quadratic sieve.
5. Let $f(x) = x^3 + ax + b$. Over $\mathbb{C}$ we have the following factorization

$$f(x) = (x - e_1)(x - e_2)(x - e_3).$$

Show that $4a^3 + 27b^2 \neq 0$ if and only if all the e_i are distinct. *Hint:* Use the fact that a polynomial f has a double root for $x = x_0$ if and only if $f(x_0) = f'(x_0) = 0$.

Part III (Computational part)

Only for students in 4MA923.

1. Let $B \in \mathbb{Z}^+$. Write a function `factorBase(B)` that gives the set of primes $p \leq B$. You can use the function `probablePrime` from previous Homework.
2. Let p be an odd prime. Implement a function `findSquareRoot(a,p)` that finds the solutions to $x^2 \equiv a \pmod{p}$ (if there exists any). The function should return a list of the (two) solutions. It should return an empty list if there are no solutions. Use a formula for finding the solutions for $p \equiv 3 \pmod{4}$. For $p \equiv 1 \pmod{4}$ you can use a brute force method to look for a solution x satisfying $1 \leq x \leq (p-1)/2$.
3. Implement a function `quadraticSieve(N,m,B)` that uses the quadratic sieve to return a list `numbersSquaredSmooth` of values of t such that $t^2 - N$ is B -smooth (for $\lfloor N \rfloor + 1 \leq t < \lfloor N \rfloor + m$). The function also needs to return a matrix (list of lists) `smoothMatrix` containing the information about the prime factorization of the B -smooth numbers (as rows) and a list `primesUsed` of the primes which were used. Here follow some hints:

- To generate a factorbase (of primes $p \leq B$) use your `factorBase` function.
- Let $s = \lfloor \sqrt{N} \rfloor + 1$. Generate the list (of m numbers)

$$s^2 - N, (s+1)^2 - N, \dots, (s+m-1)^2 - N.$$

This is the list you will sieve.

- Remove as many factors of 2 as possible from the numbers of in the list. Remember to keep track of how many factors you have remove.

- For each prime odd p use `findSquareRoot` to find the solutions to $x^2 \equiv N \pmod{p}$. It is wise to use the so called Euler's criterion to first check if there are any solutions. It is also important to keep track of the primes we have used so we do not get unnecessary large matrices in the end.

Let x_1 and y_1 be the two solutions. Then lift these solutions to solutions $(x_e$ and $y_e)$ of $x^2 \equiv N \pmod{p^e}$ for $e \geq 1$ as long there is a solution x satisfying $s \leq x < s + m$. For each new power remove a factor from number of the form $x_e + jp^e$ and $y_e + jp^e$ between s and $s + m$ (this is the actual sieving). Remember (again) to keep track of all the factors you have removed! This hint is just a recommendation you can also use a more brute force technique.

- When you have sieved away all the prime factors from primes in the factor base you will hopefully have some numbers in the list that have been sieved down to 1. Return the values of t , corresponding to the B -smooth numbers of the form $t^2 - N$, in the list `numbersSquaredSmooth` and a matrix (list of list) with the factorizations of the numbers $t^2 - N$ as rows. Return also the primes you have used (this is probably a proper subset of the factor base).

4. Now let us try your function `quadraticSieve` for some of the example of the book (HPS) and compare the results.

(a) `quadraticSieve(221,16,11)`

(b) `quadraticSieve(9788111,1700,47)`

5. Implement the function `factorQS(N,m,B)` that (hopefully) finds a factor of N by using the Quadratic Sieve.

6. Now use `factorQS`-function to find a factor of the following numbers.

(a) 9788111

(b) 9488773076569

(c) 43157855252213089

Some Algorithms

```
1 def millerRabinWitnessQ(a,n):
2     #Returns True if a is Millar-rabin witness for the compositness of n.
3     #and False if it is maybe a prime
4     #Find as many factors of 2 as possible in n-1
5     k=n-1
6     r=0
7     while k%2==0:
8         r=r+1
9         k=k//2
10    #Now n-1=k*2^r and k is odd
11    #print("k =",k)
12    #print("r =",r)
13    x=fast2Power(a,k,n)
14    if x==1:
15        return False
16    for i in range(0,r):
17        if x==n-1:
18            return False
19        x=(x**2)%n
20    return True
21
22 def probablePrimeQ(n):
23     #Runs Miller-Rabin primality test for 20 random values of a
24     for i in range(0,20):
25         a=randint(2,n-1)
26         if millerRabinWitnessQ(a,n):
27             return False
28     return True
29
30 def findPrime(lBound,uBound):
31     while True:
32         n=randint(lBound,uBound)
33         if n%2==1:
34             if probablePrimeQ(n):
35                 return n
36
37 def solveLinear(a,b,m): #Solves the linear congruence equation ax=b(mod m)
38     [d,s,t]=extendedGCD(a,m)
39     print(d)
40     solutions=[]
41     if b%d==0: #We have solutions if d divides b
42         x0=(s*(b//d))%m
43         mp=m//d
44         for i in range(d):
45             solutions.append((x0+i*mp)%m)
46     return solutions
```